

Near-Precise Parameter Approximation for Multiple Multiplications on a Single DSP Block

Ercan Kalali¹ and Rene van Leuken¹

Abstract—DSP blocks are one of the efficient solutions to implement multiply-accumulate (MAC) operations on FPGA's. However, since the DSP blocks have wide multiplier and adder blocks, MAC operations using low bit-length parameters lead to an underutilization. Hence, an efficient approximation technique is introduced. The technique includes manipulation and approximation of the low bit-length parameters based upon a Single DSP - Multiple Multiplication (SDMM) execution. The accuracy of the developed optimization technique was evaluated for different CNN weight bit precisions using the Alexnet and VGG-16 networks and the ImageNet ILSVRC-2012 dataset. The optimization can be implemented without loss of accuracy in almost all cases, while it causes slight accuracy losses in a few cases. Through these optimizations, multiple parameter multiplications are performed in a single DSP block at the cost of a small hardware overhead. As a result of our optimizations, the parameters are represented in a different format on off-chip memory, providing up to 33% compression without any hardware cost. A prototype systolic array architecture was implemented employing our optimizations on a Xilinx Zynq FPGA. It reduced the number of DSP blocks by 66.6%, 75%, and 83.3% for 8, 6, and 4-bit input variables, respectively.

Index Terms—Approximate computing, multiple multiplications, DSP blocks, FPGA, systolic array

1 INTRODUCTION

THE multiply-accumulate (MAC) operation is the main computation unit for many digital signal processing applications such as convolutional neural networks (CNN), image/video processing, and audio recognition [1], [2], [3], [4], [5]. State-of-the-art CNN algorithms consist of multiple convolutional layers, each requiring millions of MAC operations. This substantial computational complexity has led GPU and FPGA vendors to offer optimized solutions.

DSP blocks in FPGAs are one of the solutions to perform multiply-accumulate (MAC) operations efficiently. These blocks have built-in multiplier and accumulator accelerators instead of using look-up tables (LUTs) to provide high performance and low power. For example, the Xilinx DSP48E1 has a 25×18 -bit multiplier, 25-bit pre-adder, and 48-bit accumulator. Each DSP block can perform one MAC operation with high performance and low power consumption. Hence, the number of parallel MAC operations implemented is restricted given the number of available DSP blocks on the FPGA. Furthermore, DSP48 blocks are placed on multiple DSP columns on the FPGA architecture, which enables the implementation of multiple MACs efficiently by cascading multiple DSP48 blocks in each DSP column. The cascaded connection of the DSP blocks is restricted given the number of DSP48 blocks in each DSP column. There are a few studies

available in the literature on the efficient use of DSP blocks to overcome these limitations [6], [7], [8], [9], [10], [11], [12].

Since the DSP blocks have wide multiplier and adder blocks, MAC operations using low bit-length fixed-point parameters lead to an underutilization problem. We introduce an efficient parameter approximation technique that performs the Single DSP - Multiple Multiplication (SDMM) and increases the DSP utilization ratio. We start with the manipulation described in [13]. Since the manipulation in [13] is only employable for a few pre-computed constants, we first modified it to support performing fixed-point signed parameter manipulation during hardware runtime. This modification allows the dynamic control of manipulation of the millions of parameters to use for the SDMM on the FPGA. We achieve this using a table lookup technique. Further, we implemented multiple signed fixed-point parameter multiplications on a single DSP. The traditional MAC implementation in the DSP block is changed by separating multiplication and accumulation operations to implement the SDMM. While the accumulator hardware available in DSP block is used for multiple parameter multiplication, parallel LUTs are employed for the accumulation part of the MAC operation.

We introduce a novel parameter approximation technique that modifies the parameter's value to guarantee that the bit-length of the manipulated parameter is at most 3, which reduces the overhead caused by the lookup table used to store manipulated parameters significantly. It also makes it easier to multiply more parameters on a single DSP block since the approximation reduces the bit-length of the manipulated parameters. Additionally, it significantly simplifies the control complexity of the hardware implementation. To make sure that all parameter tuples can be implemented using the SDMM, a novel fine-tuning technique is implemented, which ensures the number of parameter multiplication per DSP block is fixed. Through these optimizations,

- The authors are with the Department of Microelectronics, Delft University of Technology, 2628 CD Delft, The Netherlands.
E-mail: {e.kalali-1, t.g.r.m.vanleuken}@tudelft.nl.

Manuscript received 26 April 2021; revised 23 August 2021; accepted 3 October 2021. Date of publication 11 October 2021; date of current version 9 August 2022. This work was supported in part by EU ECSEL Joint Undertaking, which funded the NewControl project under Grant 826653-2.

(Corresponding author: Ercan Kalali.)

Recommended for acceptance by G. Constantinides.

Digital Object Identifier no. 10.1109/TC.2021.3119187

multiple parameter multiplications are performed in a single DSP block at the cost of a small hardware overhead. For example, a single DSP block executes 3 8-bit fixed-point parameter multiplications.

The accuracy loss caused by the introduced approximation technique and fine-tuning is evaluated using the Alexnet [14] and VGG-16 [15] networks, and the ImageNet ILSVRC-2012 dataset [16]. Compared to quantized fixed-point implementations of the Alexnet and VGG-16, our approximation and fine-tuning techniques can be implemented without loss of accuracy in almost all cases, while it causes slight accuracy losses in a few cases.

After the multiplication packing and the parameter approximation, some pre-calculated values, which are used for single DSP - multiple parameter multiplication (SDMM), are stored on the on-chip ROM. Result of this, the actual parameter values are not necessary to store on the off-chip memory. Instead, the index values of the ROM are stored on the off-chip memory. This optimization provides up to 33% compression on the off-chip memory without any hardware cost. The compression rate can be increased by up to 96.23% and 97.03% using Huffman coding and weight pruning for the Alexnet and VGG-16 networks, respectively. Also, the compression caused by the parameter representation change compensates the on-chip memory overhead caused by the ROM. It keeps on-chip memory size the same or less compared to traditional implementations.

The main contributions of this paper are:

- We introduce a novel multiplication packing technique for multiplying one input variable with multiple constants as well as variables using a single DSP block (SDMM). In particular, we significantly reduce the usage of DSP blocks for FPGA implementations of CNN inference by packing the weights of a given CNN model using our technique.
- We propose a novel approximation technique that allows using hardware resources efficiently without sacrificing accuracy. We achieve this by limiting the bit length of manipulated parameters.
- We show a compression method that, significantly, reduces the off-chip memory transfers of parameters by using a ROM that stores pre-calculated results of our multiplication packing technique on on-chip memory. Our compression method guarantees a reduction of 33%, 25%, and 16.7% for 8, 6, and 4-bit parameters when coupled with our approximation technique.
- Finally, we design an area efficient systolic array architecture implementation. It uses 66.6%, 75%, and 83.3% fewer DSP blocks for 8, 6, and 4-bit input variables compared to the baseline FPGA implementation.

The rest of this paper is organized as follows. Background information and related works are summarized in Section 2. In Section 3, our technique is explained. Then, we describe our processing element architecture in Section 4. Section 5 explains the prototype systolic array architecture. Then, Section 6 presents the implementation results. Finally, the conclusion is given in Section 7.

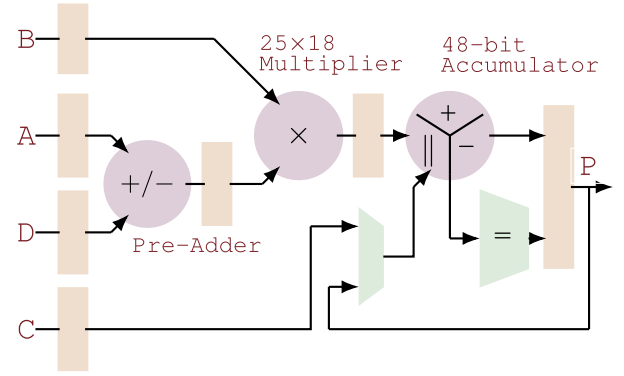


Fig. 1. Xilinx DSP48E1 architecture.

2 BACKGROUND AND RELATED WORK

2.1 DSP Blocks

Different FPGA vendors provide built-in DSP blocks in their FPGAs. Although there are some differences between DSP architectures used by different FPGA manufacturers, all of these DSP blocks include the optimized hardware blocks required to handle MAC operations efficiently. Xilinx uses DSP48E1 and DSP48E2 blocks on different FPGAs. Fig. 1 shows the Xilinx DSP48E1 architecture, which has a 25-bit pre-adder, 25×18 multiplier, and 48-bit accumulator. Xilinx DSP48E2 has a 27-bit pre-adder, 27×18 -bit multiplier, and 48-bit accumulator.

DSP48 blocks can be configured differently for different type of operations. Xilinx DSP blocks can execute MAC operation using the multiplier and the accumulator available in the DSP48E1 (DSP48E2) as in (1). Since it has a wide multiplier and accumulator, executing a MAC operation on the DSP block for reduced bit length fixed-point parameters cause underutilization issue. Using the 25×18 -bit multiplier for 8×8 -bit multiplication is an example of an underutilization problem. To fully utilize the DSP blocks, we develop an efficient multiplication packing technique, which can execute multiple parameter multiplication on a single DSP block (SDMM)

$$P = (A \cdot B) + C. \quad (1)$$

2.2 Accelerator Design : CNN Use Case

Research conducted on improving the utilization and/or performance of DSP blocks in the literature generally employs CNNs as an use-case, because CNN models require millions of MAC executions. We also implemented CNN inference on the systolic array architecture to evaluate the performance of our technique.

Convolution computations are still the main reason for the computational complexity of the CNN models, even if the state-of-the-art CNN models accommodate different layers such as pooling and activation. CNN models consist of multiple convolution layers, each containing many parallel convolution blocks. For example, VGG-16 has 13 different convolution layers, each with 64 to 512 parallel convolution channels. Also, each CNN kernel produces the result by summing the multiplication result of the CNN weight and input feature (I) with partial sums. As a result, state-of-the-art CNN models need to execute millions of multiply-accumulate

TABLE 1
Number of MAC Required for Convolutions

	Alexnet	VGG-16	GoogleNet	MobileNet
# of MAC (Millions)	666	15300	1233	568

(MAC) operations for classification. Table 1 shows the required number of MAC operations for some CNN models.

Most of the CNN models are still trained on a GPU using floating-point weight values. On the other hand, the fact that CNN inference can be realized with negligible loss of accuracy using fixed-point reduced bit length CNN weights has made FPGA and ASIC designs a strong competitor of the GPU for CNN inference implementations [17], [18], [19]. Since FPGA and ASIC implementations offer high parallelism with low-power consumption, they are used to meet the intensive computing requirement of CNN models.

A systolic array (SA) architecture usually consists of a combination of processing elements (PE) in two dimensions, each of which can execute one MAC operation [20]. It provides a low-power and high-performance solution for matrix-matrix multiplications. Since SA architecture allows data movement between neighboring PEs only, it reduces the data movement cost significantly. Also, SA architecture enables data reuse with different dataflow techniques. A SA architecture is widely used for CNN accelerators implemented on FPGA and ASIC [21], [22], [23], [24]. Google TPU [23] and Xilinx xDNN [24] are two well-known examples where SA is used for CNN inference. We demonstrated the efficiency of our optimizations on the SA hardware.

2.3 Related Work

Previous publications in the literature about efficient use of DSP blocks can be classified into 3 groups; (1) optimizations targeting higher processing throughput, (2) optimizations enabling multiple parameter multiplications on a single DSP block, (3) novel DSP designs optimized for multiplication/MAC operations, which can be used in the future FPGA architectures. Our multiplication packing technique is in the second group.

In [6], the authors investigated on the efficient use of cascade interconnection for DSP blocks to implement convolution kernels. They reorganized matrix-vector multiplication operations to make them cascade interconnection friendly, and implemented on the Xilinx UltraScale+ FPGA. Their FPGA implementation improved CNN inference latency, but it does not include an optimization to reduce the usage of DSP blocks.

A novel method was introduced in [7] to perform two MAC operations per DSP block with the concatenation of two 8-bit parameters. This method reduced the DSP block usage by 50% with the overhead of 11 LUTs and 12 FFs per MAC operation. The technique in [7] can only be used for 8-bit parameters, while our technique can support different parameter bit lengths such as 4, 6, and 8. Our multiplication packing technique uses less number of DSP blocks than the technique presented in [7]. Also, our technique has additional benefits like the compression on off-chip memory by parameter representation change.

An optimized DSP block (PIR-DSP) architecture was developed in [8] to implement reduced bit length fixed-point multiplications efficiently. They used run-time decomposable multiplier architecture and added a register file, which can also be configured as FIFO, to DSP block architecture. The PIR-DSP architecture increased the performance of the 9×9 -bit MAC operation 6 times compared to using the Xilinx DSP48E2 block. It also reduced energy consumption by 31%. Similar to [8], [9] developed a new DSP architecture based on the Intel Arria 10 DSP architecture. The DSP architecture given in [9] supports two simultaneous 8-bit parameter multiplications or four simultaneous 4-bit parameter multiplications. This increased the Arria 10 DSP size by 12%. On the other hand, it reduced the DSP block usage by 15% and 30% for 8-bit and 4-bit parameters, respectively. Instead of using DSP architecture implemented on the state-of-the-art FPGAs, novel DSP architectures, which can be used for future FPGAs, were presented in [8], [9]. Since FPGAs using these architectures are not available and the architectures of these DSP blocks are different from the existing ones, it is not possible to compare our multiplication packing technique with these implementations.

A packing technique is developed in [10] for 2- or 3-bit multiplications on DSP blocks for block floating-point implementations of DNN algorithms on FPGA. A packing technique, which can perform two 8-bit signed parameter multiplication on an 18×18 -bit multiplier, is introduced in [11] for Intel Stratix 10 FPGA. This packing technique also uses 24 extra LUTs per 8-bit signed parameter multiplication. An improved version of the packing method introduced in [11] is presented in [12]. Instead of 8-bit signed multiplication, two 7-bit unsigned parameter multiplications are executed on a single 18×18 -bit multiplier of Intel FPGAs. Sign-magnitude representation is used for 8-bit signed numbers. Twelve extra LUTs are used in [12] for each 7-bit unsigned parameter multiplication. The packing techniques developed in [11] and [12] can reach high clock speeds as they target the Intel Stratix 10 FPGA that uses Intel's 14nm tri-gate process technology.

Xilinx provides the Deep Learning Processing Unit (DPU) in [25]. The DPU is an optimized programmable framework to implement deep learning algorithms efficiently on Xilinx FPGAs. Two different optimizations are available for MAC operations on a DPU. A DSP dual data rate (DDR) technique is supported in the DPU. The DSP-DDR technique, which is used to increase the throughput of the DSP blocks, enables using a high-frequency clock signal for DSP blocks. Also, the DPU offers two different resource allocation options (low-DSP and high-DSP). In case of a low-DSP usage option, DSP blocks are allocated only for multiplication operations. In case of a high-DSP usage option, DSP blocks are allocated for both multiplication and accumulation operations. The detailed comparison of our implementation with the DPU is given in Section 6.

A method for constant multiplication packing was developed in [13]. This method reduced the bit-length of constants by a simple mathematical manipulation. Through the manipulation presented in [13], the multiple constant multiplications can be executed on a single DSP block. This method carries out the manipulation of the constants using software and loads them to DSP blocks before runtime. As a

result of this, only one tuple of constants can be executed on a DSP block during runtime. Although the technique developed in [13] works quite efficiently on hardware implementations used a small number of constants, as the number of constants to be multiplied is limited by the number of DSP blocks available in the FPGA, the increase in the number of constants significantly reduces the utilization efficiency of DSP blocks. Additionally, state-of-the-art DSP algorithms need much more multiplications than the number of DSP blocks available in the FPGA. As a consequence, it is not possible to implement state-of-the-art DSP algorithms in FPGA using the technique presented in [13].

Algorithm 1. Parameter Manipulation

```

input:  $W$ 
output:  $MW, n, s$ 
function manipulation( $W$ )
  if  $W > 0$  then
    while  $\text{mod}(W, 2) = 0$  do
       $s \leftarrow s + 1$ 
       $W \leftarrow W \div 2$ 
    end while
  end if
   $W \leftarrow W - 1$ 
  if  $W > 0$  then
    while  $\text{mod}(W, 2) = 0$  do
       $n \leftarrow n + 1$ 
       $W \leftarrow W \div 2$ 
    end while
  end if
   $MW \leftarrow W$ 
end

```

3 OUR MULTIPLICATION PACKING TECHNIQUE

DSP blocks in FPGAs have wide multiplier and accumulator hardware considering bit-lengths used in various algorithms such as state-of-the-art CNN inference hardware implementations. The aforementioned causes the underutilization problem for DSP blocks, which can be solved by performing SDMM. Xilinx introduced a white-paper [26], which offers a solution to implement two 8×8 bit multiplications per DSP block using simple concatenation and shift operations. The method given in [26] is only applicable for 2 8-bit parameters. Recently, Xilinx introduced an other white-paper [27], which offers a solution to implement three 4×4 bit multiplications per DSP block for DSP48E2.

Further, a mathematical manipulation method developed in [13] that can multiply multiple constants with one variable on a single DSP block. The method given in [13] is applicable only in cases where there are a small number of constants since the number of constants that can be multiplied by this manipulation is limited with the number of DSP blocks in the FPGA. Thus, it is not possible to implement state-of-the-art DSP algorithms, where millions of multiplications are required, using the method given in [13].

We present an efficient multiplication packing technique, which allows the SDMM execution, based upon the constant manipulation technique in [13]. First, in Section 3.1,

we briefly explain the constant manipulation. Then, we introduce our novel parameter approximation technique in Section 3.2. Finally, our multiplication packing technique is described in Section 3.3.

I, W, MW , and MW_A denote the input variable, fixed-point parameter, manipulated parameter, and manipulated approximate parameter. v and c denote the bit lengths of the input variable and the fixed-point parameter.

3.1 Parameter Manipulation

Xilinx DSP blocks have one multiplier and one accumulator hardware. As shown in (1), DSP block can be configured for a MAC operation using one multiplier and one accumulator. We employ this option to configure our SDMM operation instead of a MAC operation. We executed multiple multiplications using a multiplier and an accumulator available in the DSP block. Accordingly, as shown in (2), resources of the DSP block can be fully utilized by restructuring the multiplicand such that it consists of a multiplication and addition operations. This reduces the bit length of the multiplicand and shares the multiplication between the multiplier and accumulator hardware of the DSP block. The n and s values are determined to minimize MW as shown in Algorithm 1

$$W = 2^s \cdot (1 + 2^n \cdot MW). \quad (2)$$

The multiplication of the parameter (W) with the input variable (I) can be written as in (3). Eq. (3) uses both the multiplier and accumulator hardware of the DSP block to perform a single multiplication operation. Eq. (3) makes multiple parameter multiplication on a single DSP block possible, since the multiplication operation is shared between the multiplier and accumulator hardware within the DSP block. The multiple multiplications are achieved by the concatenation of manipulated parameters for the multiplier and accumulator inputs of the DSP block

$$P = W \cdot I = I \cdot 2^s \cdot (1 + 2^n \cdot MW). \quad (3)$$

3.2 Our Novel Parameter Approximation Technique

The number of parameters (k), which can be multiplied on a single DSP block depends on the bit-lengths of the input variable (v) and manipulated parameters ($c_i - (s_i + n_i)$). We set the k number for the SDMM to 3, 4, and 6 for 8, 6, and 4-bit input variables, respectively.

The hardware implementation cost of the constant manipulation given in Algorithm 1 is higher than the implementing MAC operation using LUTs. As a solution, we design a Table Look Up based implementation, which stores the n and s values and the multiplicand input of the DSP block required for manipulated multiple parameter multiplication. The aforementioned Look-Up Table architecture is implemented on on-chip ROM and used as a dictionary for the SDMM. Depending on the input variable bit length, this Look-Up Table may need up to $((2^v)^k)$ different entries. This leads to MBs of on-chip ROM overhead, which is quite large for the on-chip storage capacity of the state-of-the-art FPGAs. As a solution, we introduce a novel parameter approximation technique to reduce the size of the on-chip ROM.

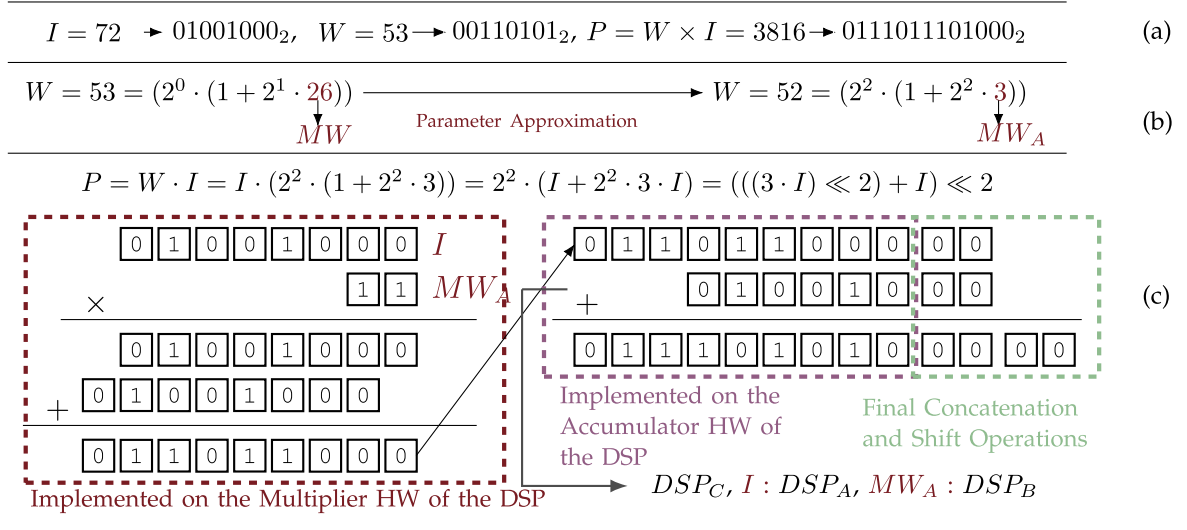


Fig. 2. Numeric example of parameter manipulation, (a) Exact multiplication, (b) Parameter approximation, and (c) Manipulated multiplication.

Our approximation technique gives a constraint over the mathematical manipulation equation given in (2). The approximated version of (2) is shown in (4). This approximation limits the bit length of MW to a maximum of 3, regardless of the W . Although the value of MW is limited to a maximum of 3 bits, 128 of 256 8-bit signed parameters can be implemented without any error thanks to n and s . The other 128 values can be implemented with minor changes. The parameter approximation reduces the number of maximum different entries for the Look-Up Table to 8192, 16384, and 16384 for 8, 6, and 4-bit parameters, respectively. These numbers are found by software simulations

$$W = 2^s \cdot (1 + 2^n \cdot MW_A) \quad \forall MW_A \in (0, 1, 3, 5, 7). \quad (4)$$

3.3 Our Multiplication Packing Technique

The multiplication packing technique separates MAC operation as multiplication and accumulation. Since the hardware cost of an accumulator is less than a multiplier, accumulation operation can be implemented using LUTs with a small hardware cost. Thanks to this, multiplier and accumulator hardware available in the DSP block can be used for the SDMM. The proposed multiplication technique is explained in 4 steps for simplicity; (1) multiplication packing for single parameter, (2) multiplication packing for signed input variable, (3) multiplication packing for multiple parameters, and (4) fine-tuning on parameter tuples.

3.3.1 Multiplication Packing for Single Parameter

The input variable (I) is multiplied with the approximate manipulated parameter as shown in (5). Since the bit length of MW_A is up to 3, multiple multiplications of the MW_A with I is possible by concatenating these multiplications on the multiplier of the DSP block. The remaining part of the multiplication with the parameter is executed on the accumulator hardware of the DSP block. As in (5), the multiplication result ($MW_A \cdot I$) is shifted n -bits before the addition. This means that the least significant n -bits of the multiplication result are always zero. Hence, instead of performing addition on the least significant n -bits, the least significant

n -bits of the I is concatenated directly to the addition result, which reduces the cost of the addition

$$\begin{aligned} W \cdot I &= (2^s \cdot (1 + 2^n \cdot MW_A)) \cdot I \\ &= (I + ((MW_A \cdot I) \ll n)) \ll s. \end{aligned} \quad (5)$$

A numeric example is shown in Fig. 2. Fig. 2a shows the traditional multiplication of I and W . As shown in Fig. 2b, the bit length of the MW is reduced 5 to 2 with a small change in W thanks to our approximation technique. The multiplication of the manipulated approximate parameter with the input variable is shown in Fig. 2c. The MW_A is multiplied with I using the multiplier hardware available in the DSP block. The MSBs of the multiplication result is accumulated with the MSBs of the I using the accumulator hardware available in the DSP block. Finally, concatenation and shift operations are implemented at the output of the DSP block.

3.3.2 Signed Multiplication With Manipulation

The aforementioned technique can perform a signed multiplication with the signed input variable. Still, some modifications are necessary on the multiplication packing technique for signed multiplication. Since multiple parameter multiplication is performed on a DSP block by concatenating multiple manipulated multiplications, the multiplier hardware in the DSP block performs multiplication by ignoring the addition of the sign extension part. Subsequently, the addition of sign extension bits are executed separately, and the result of the sign extension is added to the multiplication result in an intelligent way using accumulator hardware on the DSP block. That prevents the multiplication operation from using extra bits caused by the sign extension and makes possible multiplications of more parameters on a single DSP block.

In case of multiplication packing of exact parameters, sign extension bits for different parameters are calculated by using (6). After that, SE_x should be concatenated with $I[v-1:n]$ to determine the value that is used for the accumulation

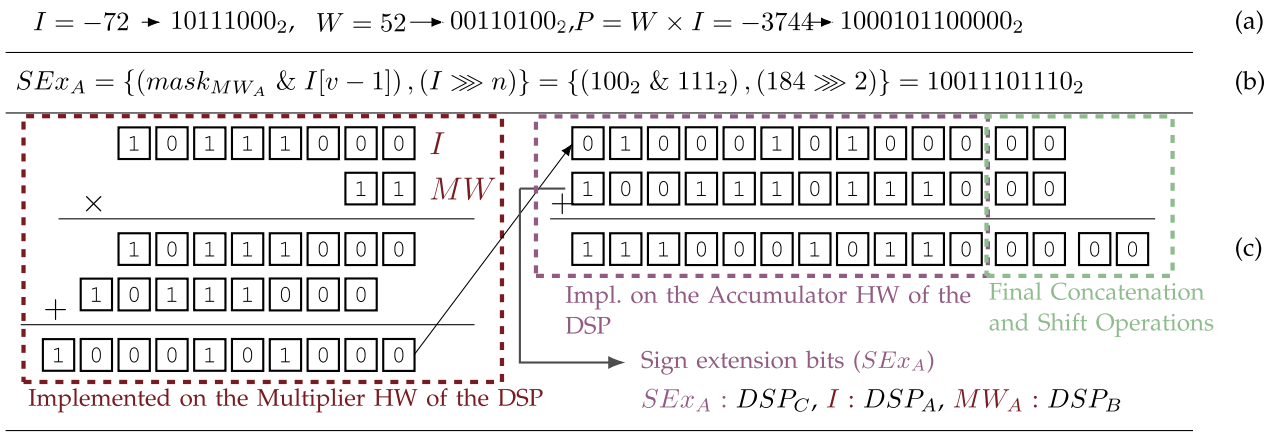


Fig. 3. Numeric example of parameter manipulation (Signed input), (a) Signed multiplication, (b) Generation of sign extension bits, and (c) Signed multiplication using the multiplication packing.

$$Sex = (I[v-1] \cdot (2^{(m-s)} - W \cdot 2^{-s}))[(c-s-1) : 0]. \quad (6)$$

Whereas, the proposed parameter approximation technique, significantly, simplifies the calculation of sign extension bits as shown (7). Also, the value calculated in (7) is used for accumulation directly. It does not need extra concatenation. The $mask_{MW_A}$ equals to 111₂, 110₂, 100₂, 010₂, and 000₂ for the $MW_A = 0, 1, 3, 5$, and 7 , respectively

$$Sex_A = \{(mask_{MW_A} \& I[v-1]), (I \ggg n)\}. \quad (7)$$

A numeric example of a signed multiplication with the proposed multiplication packing is given in Fig. 3. Fig. 3a shows the multiplication of the signed I and W . Fig. 3b generates the sign extension bits (Sex_A) using (7). Fig. 3c presents the signed multiplication.

3.3.3 Accuracy Evaluation of Approximate Signed Multiplications

We evaluated the accuracy of our multiplication packing technique, which includes our new parameter approximation for single parameters of 4, 6, and 8 bits. New metrics are defined in [28] to evaluate the arithmetic accuracy of approximate adders/multipliers. We calculated the normalized mean error distance (NMED) and mean relative error distance (MRED) metrics defined in [28]. According to the results shown in Table 2, our approximation technique can work without any error for 4-bit parameters. Since Eq. (4) can implement signed parameters smaller than 6-bits without any error, multiplication of parameters smaller than 6-bits can be performed without any error using the parameter approximation technique. It causes negligible losses for 6- and 8-bit parameters. We also compared our approximation technique with the approximate multiplier designs in the literature [29], [30], [31], [32], [33], [34]. The results show that our approximation technique outperforms the approximate multiplier designs in [29], [30], [31], [32], [33], [34].

3.3.4 Manipulation for Multiple Parameters

First, all parameters to be multiplied on a single DSP block are manipulated using (4). Afterward, depending on the

input variable bit length, parameter tuples are generated for the SDMM. For example, each parameter tuple includes 3 parameters for the 8-bit input variable. The parameters within the given parameter tuple are concatenated as shown in (8). The second row of the (8) is executed on the multiplier hardware available in the DSP block, while the third row is executed on the accumulator hardware available in the DSP block. Since the (8) includes sign extension calculations, it also supports signed multiplication. Final concatenation (with $I[n_i : 0]$) and shift (with s_i) operations are not shown in (8) for simplicity

$$\begin{aligned} & \{W_k, \dots, W_2, W_1\} \cdot I = \\ & I \cdot \sum_{i=2}^k (MW_{A_i} \ll ((i-1) \cdot (v+3)) + MW_{A_1}) \\ & + \sum_{i=1}^k (Sex_{A_i} \ll ((i-1) \cdot (v+3))). \end{aligned} \quad (8)$$

Eq. (8) is the simplified version of the exact version of the multiple manipulated parameter multiplication due to the novel parameter approximation technique. If the approximation technique is not used, the value of $(c_i - (n_i + s_i))$

TABLE 2
Accuracy Evaluation and Comparison of Approximate Signed Multiplications

Design	Bit-Width	NMED	MRED
Our	8	0.0009	0.005
	6	0.0001	0.004
	4	0.0000	0.000
[29]	8	0.0261	0.049
[30]	8	0.0035	1.988
	6	0.0063	2.658
	4	0.0106	2.773
[31]	8	0.0016	0.012
[32]	8	0.0051	0.091
[33]	8	0.0272	0.080
[34]	8	0.0234	0.134

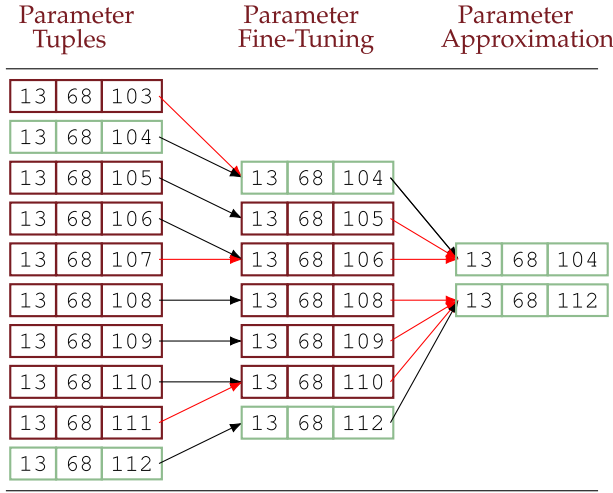


Fig. 4. Numeric example of the parameter approximation and fine-tuning (Red Arrow : Approximated).

would have to be calculated for each parameter within the parameter tuple, instead of 3 given in the second and third lines of (8). The novel approximation ensures that the extra overhead caused by these calculations on the hardware is eliminated. In addition, it is necessary to use (6) instead of (7) for the sign extension, when the proposed approximation technique is not used.

3.3.5 Fine-Tuning of Parameter Tuples

The input variable bit length (v) is the major term to determine the number of parameters that can be multiplied on a single DSP block. Whereas, depending on the parameter values, it is not always possible to perform a SDMM with this number. Thus, a fine-tuning of parameters is performed to guarantee that the number of parameter multiplication per DSP block is fixed. Unlike the traditional techniques in which each parameter is independently tuned, our technique performs fine-tuning on parameter tuples.

Our fine-tuning technique consists of three steps. First, the number of parameters that can be multiplied on a single DSP block is determined based on the bit-lengths of the input variable and the parameters. 3, 4, and 6 parameters can be multiplied on a single DSP block for the 8, 6, and 4-bit input variables (I), respectively. Later, all of the parameter tuples that can be multiplied on a single DSP block is determined using the fixed number found in the first step. Finally, the parameter tuple, which cannot be multiplied on a single DSP block, is replaced by the closest parameter tuple in the set determined in the second step. The Bray-Curtis distance formula, which is given in (9), is used to determine the closest parameter tuple

$$Cu : 1D \text{ Array}, \quad v : 1D \text{ Array} \quad (9)$$

$$BC = \sum ||u_i| - |v_i|| / \sum |u_i + v_i|.$$

A numeric example of the fine-tuning and parameter approximation is shown in Fig. 4. Ten different parameter tuples are shown in the left column of the Fig. 4. The parameter tuples, which cannot be multiplied on a single DSP block, are tuned using (9). The fine-tuning step reduces the number of unique parameter tuples to seven. Furthermore, according to the approximation rule presented in (4), some of the parameters are approximated. The parameter approximation reduced the number of unique parameter tuples to two.

Accuracy of the parameter approximation and fine-tuning techniques for 4, 6, and 8-bit fixed-point parameters and 4, 6, and 8-bit input variables (I) were evaluated on the Alexnet and VGG-16 networks. Validation images of the ImageNet ILSVRC-2012 were used to measure accuracy results. The error increase caused by the approximation and fine-tuning is reported in Table 3 compared to a quantized implementation of the Alexnet and VGG-16. As shown in Table 3, the parameter approximation and fine-tuning can decrease the error in some cases, while it causes a slight increase in the error in other cases. Since Eq. (4) can implement signed parameters smaller than 6-bits without any error, multiplication of parameters smaller than 6-bits can be performed without any error using the parameter approximation technique.

4 PROCESSING ELEMENT (PE) ARCHITECTURE

Our PE architecture is different from the traditional PE architectures implemented for MAC operation. Traditional PE implementations can execute one MAC operation per DSP block using the multiplier and accumulator hardware available in the DSP block. Our PE architecture is customized for the SDMM execution. The multiplication and the accumulation operations are executed separately in the implemented PE architecture. The multiplier and accumulator hardware available in the DSP block is configured to execute multiple parameter multiplications. Corresponding accumulations of these multiple parameter multiplications are executed using the LUTs available in the FPGA. This brings up two advantages. First, a wide multiplier hardware available in the DSP block can be fully-utilized for the reduced bit-length parameters. Second, it enables an efficient resource sharing in FPGAs with a limited number of DSP resources.

The PE architecture is shown in Fig. 5 for the 8-bit parameters and the 3 parameter multiplications per DSP block. The PE architecture is used by scaling for different bit lengths. This PE architecture consists of four parts; parameter decompression, multiple parameter multiplication, post-processing, and accumulation.

TABLE 3
Error Increase (%) Caused by the Approximation Technique (CNN Use Case)

CNN Model	(W,I) : Bit Lengths of CNN Weight and Input Variable								
	(8,8)	(8,6)	(8,4)	(6,8)	(6,6)	(6,4)	(4,8)	(4,6)	(4,4)
Alexnet	0.06	0.02	-0.24	-0.01	-0.11	0.05	0.00	0.00	0.00
VGG-16	-0.04	0.02	-0.11	-0.01	-0.05	-0.03	0.00	0.00	0.00

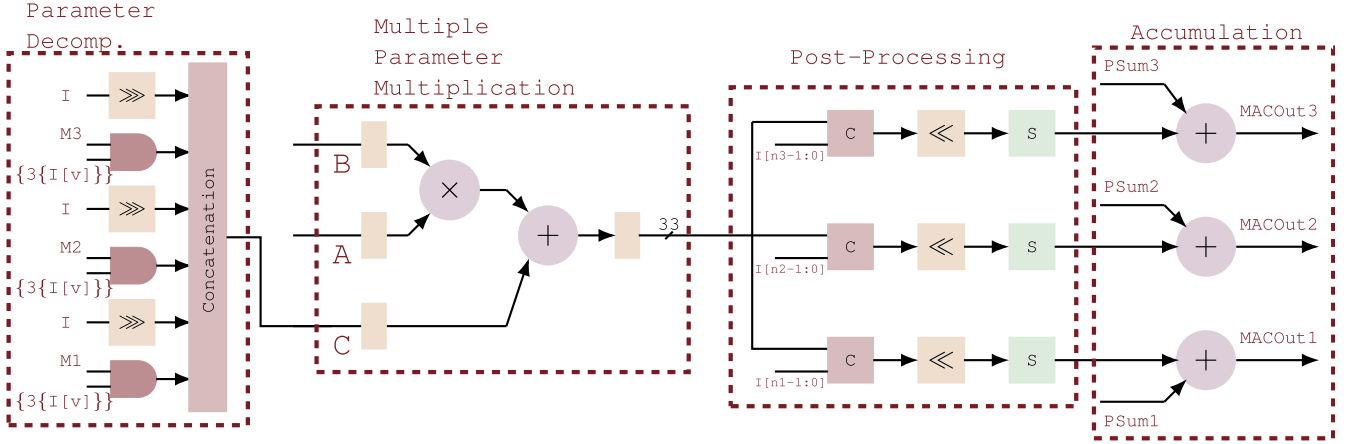


Fig. 5. Our PE architecture.

In the parameter decomposition part, the parameter decomposition is performed using the ROM output. Since the calculation of the multiplicand input of the DSP block is independent of the input variable as shown in the second line of the (8), the multiplicands ('A' input of the DSP block) are stored in the ROM for different parameter tuples. Also, n and s values for each parameter within the parameter tuple are stored in the ROM. The most significant 24 bits of the output of the ROM, which stores the multiplicand given in the second line of the (8), are connected to the 'A' input of the DSP block directly. The least significant bits of the WROM output is used to store the shift values required to generate the 'C' input of the DSP block. The 'C' input of the DSP block is generated using the parameter decomposition part shown in the Fig. 5. Masks (M1, M2, and M3), which are used to generate the sign extension bits, are stored in the LUTs, and only 30 6-input LUTs are necessary for the entire hardware. The parameter decomposition hardware needs 35 LUTs for each 3 parameter multiplications (for 8-bit fixed-point parameters). Also, the LUT overhead caused by the parameter decomposition for the entire hardware is reported in Section 4.

Unlike the traditional PE implementations, SDMM (3 multiplications per PE for 8-bit fixed-point parameters) is executed in the second part of the design. This achieved using the multiplier and accumulator hardware available in the DSP block.

The Eq. (10) shows the 'A', 'B', and 'C' inputs of the DSP block using (8). Each part is used as the input of the DSP block. The second row of the (8) is given to the 'A' input (25-bit) of the DSP block. The input variable is sent to the 'B' input (18-bit) of the DSP block. The 'C' input (48-bit) of the DSP block takes the third row of (8). The 'A' input (25-bit) of the DSP block is multiplied with the 'B' input (18-bit) of the DSP block, and the result of the multiplication is accumulated with the 'C' input of the DSP block.

The post-processing part (Fig. 5) takes the output of the DSP block as input and split it into three parts for 3 parameter multiplications. Final concatenation (with $I[n-1:0]$) and shift operations ($\ll s$) are employed for each part in parallel. Subsequently, S blocks perform sign conversion using the sign bits of parameters

$$A = \sum_{i=2}^k (MW_{A_i} \ll (v + (i-1) \cdot (v+3))) + MW_{A_1}$$

$$B = I$$

$$C = \sum_{i=1}^k (SEx_{A_i} \ll (i-1) \cdot (v+3)).$$

(10)

Finally, accumulations are performed with the multiplication results to calculate the results of multiple MAC operations. LUTs are used for the final accumulation.

5 SYSTOLIC ARRAY ARCHITECTURE

A top-level architecture of the prototype systolic array (SA) hardware is shown in Fig. 6. Since the CNN models are employed as a use case, the designed systolic array architecture is customized for CNN inference. Four different memory blocks are implemented for on-chip data storage. In addition, on-chip ROM architecture is used as a dictionary to generate manipulated DSP inputs. 4, 6, and 8 bit signed fixed-point parameter multiplications are supported. Depending on the parameter bit length and the input variable bit length, each processing element (PE) is designed to execute 3, 4, or 6 parameter multiplications per DSP block.

Four AXI-mapped memory blocks, which have multiple BRAMs, are used for on-chip data storage. OMem (Fig. 6)

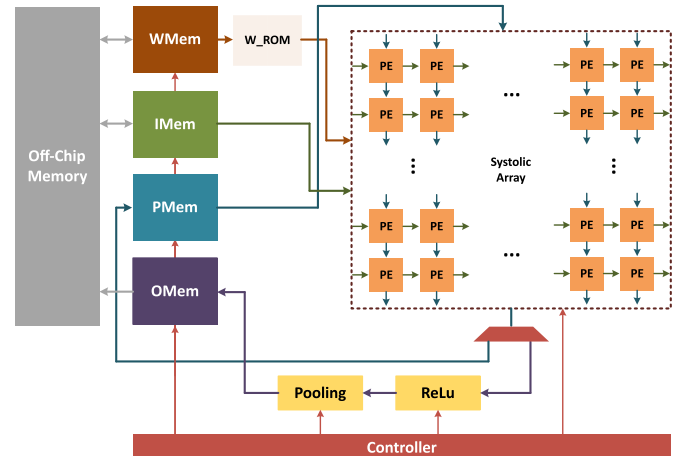


Fig. 6. Systolic array architecture.

TABLE 4
Compression Rates (CONV Layers)

(W,I)	CNN Model		DC [35]	H	WRC	WRC + H	P + WRC + H
(8,8)	Alexnet	Conv	9.09% (11.0×)	14.65% (6.8×)	66.6% (1.5×)	10.80% (9.3×)	8.96% (11.2×)
	VGG-16	Conv	7.28% (13.7×)	14.18% (7.0×)	66.6% (1.5×)	10.17% (9.8×)	8.49% (11.8×)
(6,6)	Alexnet	Conv	—	8.73% (11.5×)	75.0% (1.3×)	6.71% (14.9×)	6.07% (16.5×)
	VGG-16	Conv	—	8.10% (12.3×)	75.0% (1.3×)	6.10% (16.4×)	5.64% (17.7×)
(4,4)	Alexnet	Conv	—	3.67% (27.2×)	83.3% (1.2×)	4.26% (23.5×)	3.07% (32.6×)
	VGG-16	Conv	—	3.29% (30.4×)	83.3% (1.2×)	3.77% (26.5×)	2.97% (33.6×)

(W,I): Bit Lengths of W and I, DC: Deep Compression, H: Huffman Coding, WRC: Parameter Representation Change, P: Pruning.

store output results before sending them off-chip memory. PMem stores partial sums to reuse it during convolution operations. IMem stores the input values (I) for convolution multiplications. Unlike the other 3 memory blocks, WMem stores the address values for WROM.

Since the 'A' input value is independent of the input variable as shown in (10), it is possible to calculate the 'A' values for parameter tuples once and store them on the on-chip ROM. Notice, the shift values, which are necessary for the calculation of the 'C' input, do not change with the input variable (I). Thus, these shift values are also calculated once and stored on the on-chip ROM. These shift values configure the hardware designed to generate the 'C' input of the DSP block. Storing all the required values for all parameter tuples brings the MBs of on-chip memory overhead, which is not tolerable considering the on-chip data storage capacity of the state-of-the-art FPGAs. A solution to this memory overhead problem has been achieved thanks to our parameter approximation technique. Since the developed parameter approximation technique constraints the MW s, it also reduces the number of possible parameter tuples. This makes the memory overhead problem manageable. As shown in Fig. 4, instead of storing the required values for ten different parameter tuples, the required values are stored for only two different parameter tuples thanks to our parameter approximation technique.

Since the WROM stores, the 'A' input values, and the shift values required to generate the 'C' input values, all the information required to perform the parameter multiplication can be obtained from WROM. Consequently, there is no need to store actual parameters on the off-chip memory and the on-chip WMem. Instead, each parameter tuple can be represented as an index value of the WROM in the off-chip memory and on-chip WMem. The index values stored in the off-chip memory and the on-chip WMem consist of the address of the WROM and the sign bits of the parameters in the parameter tuple. For example, a 16-bit address value is stored for each parameter tuple consisting of 8-bit fixed-point parameters. The most significant 13-bits are used to index the WROM, while the least significant 3-bit stores the sign bits of the 3 parameters. This enables 3x8-bit parameters to be represented with 16-bit in the off-chip memory and the on-chip WMem. The parameter representation change provides 33% compression (Compression Rate: 66.6%) of the parameters and reduces the access rate to the off-chip memory by a third without any hardware overhead. It is also possible to apply other compression

techniques such as Huffman coding and pruning combined with the parameter representation change (WRC). Table 4 shows the compression performance of the Alexnet and VGG-16 networks using parameter representation change, Huffman coding, and pruning. These results were also compared with the Deep Compression [35].

As shown in Table 4, our technique combined with Huffman coding and pruning produces comparable results with the Deep Compression. Only WRC was implemented in the SA hardware. Compression results obtained using the Huffman coding and pruning were reported for analysis and comparison.

Even though the approximation technique reduces the WROM size, it is still an overhead for the hardware implementation. However, the size of the WMem implemented in this hardware is less than the traditional implementations because parameters are represented on the WMem by fewer bits due to parameter representation change. This may compensate for the overhead caused by WROM. As shown in Fig. 7, the number of parameters stored in on-chip memory with our hardware is higher than the traditional hardware implementations in case of the on-chip memory size is higher than a certain value. This analysis shows that using WROM may provide advantage instead of being overhead for hardware implementation. The on-chip memory sizes are given in Fig. 7 for traditional hardware implementations and the proposed hardware in this paper (for 4, 6, and 8 bit parameters). The initial points for our implementation in Fig. 7 shows the overhead caused by the WROM. After these initial overhead values, WMem stores the parameters.

Figs. 8a and 8b show two different PE architectures. Fig. 8a shows a traditional PE architecture, which executes a 1 MAC operation per DSP block. The PE architecture, which is shown in Fig. 8b, can execute 2 8-bit parameter multiplications on a single DSP block, while it uses LUTs and FFs for the accumulation execution. PE architecture shown in Fig. 8b uses the method presented in [26] to execute 2 multiplications per DSP block. The prototype systolic array hardware is also implemented using the PE architecture given in Figs. 8a and 8b to make a comparison with the PE architecture given in Fig. 5.

Weight stationary (WS) dataflow is used for the prototype SA architecture. Weights are initially loaded to PEs, and the inputs and Psums flow between the PEs. Since each input is multiplied with different CNN weights for different kernels in each channel, multiple CNN weights can be multiplied with one input. WS dataflow allows the reuse of the

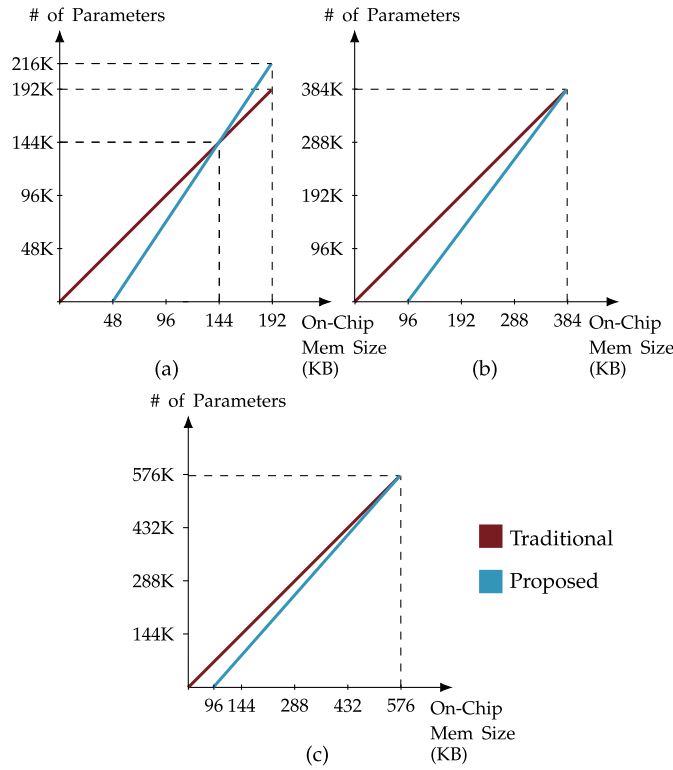


Fig. 7. On-chip memory size analysis, (a) 8-bit parameters, (b) 6-bit parameters, (c) 4-bit parameters.

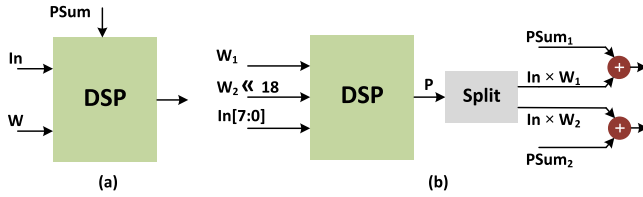


Fig. 8. PE architecture (a) One MAC per DSP, (b) Two multiplications per DSP.

CNN weights in the PEs during convolution operations. The goal of using the WS dataflow is to reduce power consumption by reducing the switching on the parameter decomposition hardware.

6 IMPLEMENTATION RESULTS

The prototype systolic array (SA) architecture is synthesized, placed, and routed for a Xilinx Zynq-7000 ZC706 FPGA using Xilinx Vivado 18.3. Three different parameter bit lengths (4, 6, and 8) are supported in the implemented SA architecture. The implementation results are given in Table 5 for the 12×12 SA. Since the number of parameters that can be multiplied on a single DSP block changes based on the input variable bit length, the number of DSP blocks reported in Table 5 is different for the different bit lengths. The number of DSP blocks given in Table 5 indicates that as the input variable bit-length decreases, our technique can perform much more multiplications on a single DSP block. The number of LUTs required for the accumulation and the overhead caused by the parameter decomposition are also reported in Table 5.

Three different versions of the SA prototype are implemented using different PE architectures; (a) One MAC per

TABLE 5
Implementation Results (12×12 PEs)

		4-bit (6M/DSP)	6-bit (4M/DSP)	8-bit (3M/DSP)
FPGA		Xilinx Zynq	Xilinx Zynq	Xilinx Zynq
LUT	P Decomp.	432	972	1680
	Post-P.	576	2016	3769
	Accum.	1152	1728	2160
DFF		5732	7667	9244
DSP		24	36	48
BRAM		54	68.5	69
Freq.		250	250	250

TABLE 6
Hardware Comparison (12×12 PE)

Bit Length	Impl.	LUT	DFF	DSP	BRAM	Freq.
4	1M	235	10167	144	48	270
	MP	2356	5732	24	54	250
6	1M	382	11189	144	69.5	256
	MP	5459	7667	36	68.5	250
8	1M	475	11973	144	92	250
	2M	2773	8343	72	92	250
	MP	8217	9244	48	69	250

DSP block (1M), (b) two parameter multiplications per DSP block (2M) and (c) multiple parameter multiplications per DSP block (SDMM) including the novel approximation technique and the parameter representation change (multiplication packing - MP). Table 6 shows the implementation results for the three different PE architectures. The implementation results for 2M are given only for 8-bit parameters because 2M can support only a multiplication with 8-bit parameters. As shown in Table 6, the SA hardware including the parameter approximation, multiplication packing (SDMM), and the fine-tuning techniques reduced the number of DSP blocks used in the baseline FPGA implementation (1M) by 66.6%, 75%, and 83.3% for 8, 6, and 4-bit implementations, respectively. We also synthesized 1M, 2M, and MP implementations of 8-bit parameters on Xilinx Zybo Z7-10 to analyze the efficiency of our design on low-cost FPGAs. As shown in Fig. 9, our MP implementation only uses 60% of available DSP blocks while 1M could not fit into the Zybo Z7-10 FPGA board.

Alternative packing techniques for 8-bit signed parameter multiplications are developed in [11], [12], [26]. The packing techniques developed in [11], [12] can pack two 8-bit signed parameter multiplications on an 18×18 -bit multiplier available in Intel's DSP block. [26] (2M) can pack two 8-bit signed parameter multiplications per DSP block on Xilinx FPGAs. On the other hand, our MP design can perform three 8-bit parameter multiplications per DSP block on Xilinx FPGAs. Accordingly, our MP design uses 33% less DSP blocks in any case compared to [11], [12], [26]. [11] and

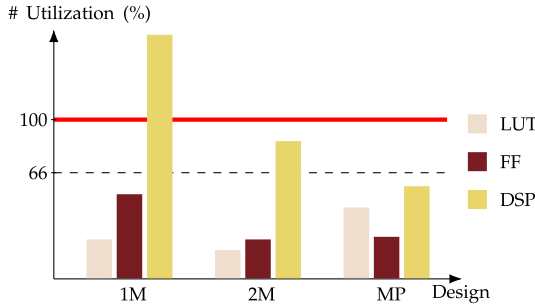


Fig. 9. FPGA resource utilization analysis (Zybo Z7-10).

[12] requires 24 and 12 extra LUTs per 8-bit signed parameter multiplication, respectively. Our MP design requires 12 extra LUTs per 8-bit signed parameter multiplication. [26] (2M) uses fewer LUTs than our technique, as reported in Table 6.

The SA hardware using our technique is also compared with the Xilinx DPU as shown in Table 7. the DPUH and DPUL refer to a high and low DSP usage configurations of the DPU. As shown in Table 7, the MP uses a fewer number of DSP blocks than the Xilinx DPUH. Also, the DPUH uses more LUTs and FFs compared to the MP. In the DPUH, MAC operations are shared between the LUTs/FFs and the DSP blocks during hardware synthesis. Also, in the DPUH, one DSP block can execute one MAC operation only. On the other hand, only multiplication operations are executed on the DSP blocks in the DPUL. Similar to the DPUH, the DPUL shared the multiplication operations between the LUTs/FFs and the DSP blocks. Accumulation operations are executed only on the LUTs/FFs. As a result, the DPUL uses a fewer number of DSP blocks than the MP, but it needs more than twice the LUTs as the MP.

We also compare the power consumption of the 1M, 2M and MP for 4, 6, and 8-bit signed parameters. The MP can implement 6, 4, and 3 multiplications on a single DSP block for 4, 6, and 8-bit signed parameters, respectively. Hence, we implemented 6, 4, and 3 MAC calculation blocks to estimate the power consumption of the 1M, 2M and MP for 4, 6, and 8-bit signed parameters, respectively. The Xilinx Vivado tool is used to estimate the power consumption of the 1M, 2M, and MP for 4, 6, and 8-bit signed parameters. All switching activities are stored in SAIF files during post-implementation timing simulation. The Xilinx Vivado tool read these SAIF files and estimates the power consumption of the 1M, 2M, and MP for 4, 6, and 8-bit signed parameters. As shown in Fig. 10, our MP implementation reduces the power consumption of the 1M by 64.1%, 54.8%, and 36% for 4, 6, and 8-bit signed parameters, respectively. Also, our MP implementation reduces the power consumption of the 2M by 30.0%, 33.3%, and 27.3% for 4, 6, and 8-bit signed parameters, respectively.

TABLE 7
Hardware Comparison With Xilinx DPU (256 PE)

Impl.	LUT	DFF	DSP	BRAM	Peak Perf. (GOPs)
DPUH	20055	28849	98	69.5	102
DPUL	21171	33572	66	69.5	102
MP	11562	13882	88	76	128

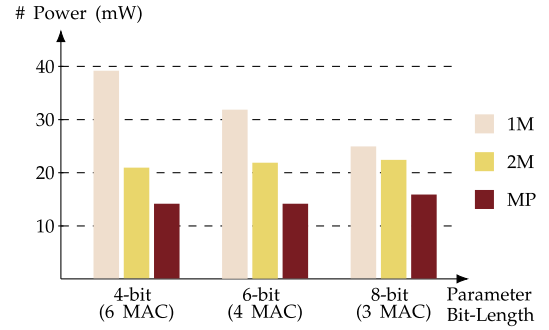


Fig. 10. Power consumption comparison.

7 CONCLUSION

In this paper, an efficient parameter approximation technique was introduced to pack multiplication operations and perform multiple parameter multiplications on a single DSP block (SDMM). This was achieved by redeployment of the accumulator component of the traditional MAC operation and insertion of a parallel addition component. This method performs a mathematical manipulation on low bit-length fixed-point parameters and reduces their bit-lengths using an efficient approximation. Also, it employs a fine-tuning step to guarantee that the number of parameter multiplications per DSP block is fixed. Accuracy of the newly presented parameter approximation technique was measured using the Alexnet and VGG-16 networks and the ImageNet ILSVRC-2012 dataset for different bit lengths. This optimization leads to minor increases or decreases in the accuracy of reduced bit length implementation of the Alexnet and VGG-16. A prototype systolic array architecture, which used a processing element that can execute multiple parameter multiplications per DSP block, was implemented on a Xilinx Zynq-7000 ZC706 FPGA. This prototype reduced the number of DSP blocks used in the baseline FPGA implementation by 66.6%, 75%, and 83.3% for the 8, 6, and 4-bit input variables. Additionally, 33% compression was achieved by changing the parameter representation.

REFERENCES

- [1] A. Krizhevsky, I. Sutskever, and G. E. Hinton, "ImageNet classification with deep convolutional neural network," in *Proc. Int. Conf. Neural Inf. Process. Syst.*, 2012, pp. 1106–1114.
- [2] V. Sze, Y. H. Chen, T. J. Yang, and J. S. Emer, "Efficient processing of deep neural networks: A tutorial and survey," *Proc. IEEE*, vol. 105, no. 12, pp. 2295–2329, Dec. 2017.
- [3] L. Deng, G. Hinton, and B. Kingsbury, "New types of deep neural network learning for speech recognition and related applications: An overview," in *Proc. IEEE Int. Conf. Acoust. Speech Signal Process.*, 2013, pp. 8599–8603.
- [4] X. Zhang *et al.*, "High-performance video content recognition with long-term recurrent convolutional network for FPGA," in *Proc. IEEE Int. Conf. Field Programmable Log. Appl.*, 2017, pp. 1–4.
- [5] O. A. Hamid, A. R. Mohamed, H. Jiang, L. Deng, G. Penn, and D. Yu, "Convolutional neural networks for speech recognition," *IEEE/ACM Trans. Audio, Speech, Language Process.*, vol. 22, no. 10, pp. 1533–1545, Oct. 2014.
- [6] A. Samajdar, T. Garg, T. Krishna, and N. Kapre, "Scaling the cascades: Interconnect-aware FPGA implementation of machine learning problems," in *Proc. 29th Int. Conf. Field Programmable Log. Appl.*, 2019, pp. 342–349.
- [7] S. Lee, D. Kim, D. Nguyen, and J. Lee, "Double MAC on a DSP: Boosting the performance of convolutional neural networks on FPGAs," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 38, no. 5, pp. 888–897, May 2019.

- [8] S. R. Rasoulinezhad, H. Zhou, L. Wang, and P. H. W. Leong, "PIR-DSP: An FPGA DSP block architecture for multi-precision deep neural networks," in *Proc. IEEE 27th Int. Symp. Field-Programmable Custom Comput. Mach.*, 2019, pp. 35–44.
- [9] A. Boutros, S. Yazdanshenas, and V. Betz, "Embracing diversity: Enhanced DSP blocks for low-precision deep learning on FPGAs," in *Proc. 28th IEEE Int. Conf. Field Programmable Log. Appl.*, 2018, pp. 35–357.
- [10] J. Fowers *et al.*, "A configurable cloud-scale DNN processor for real-time AI," in *Proc. ACM/IEEE 45th Int. Symp. Comput. Archit.*, 2018, pp. 1–14.
- [11] M. Langhammer, B. Pasca, G. Baekler, and S. Gribok, "Extracting INT8 multipliers from INT18 multipliers," in *Proc. 29th Int. Conf. Field Programmable Log. Appl.*, 2019, pp. 114–120.
- [12] M. Langhammer, S. Gribok, and G. Baekler, "High density 8-bit multiplier systolic arrays for FPGA," in *Proc. IEEE 28th Int. Symp. Field-Programmable Custom Comput. Mach.*, 2020, pp. 84–92.
- [13] A. C. Mert, H. Azgin, E. Kalali, and I. Hamzaoglu, "Efficient multiple constant multiplication using DSP blocks in FPGA," in *Proc. 28th IEEE Int. Conf. Field Programmable Log. Appl.*, 2018, pp. 331–3313.
- [14] A. Krizhevsky, "One weird trick for parallelizing convolutional neural networks," 2014, *arXiv:1404.5997*.
- [15] K. Simonyan and A. Zisserman, "Very deep convolutional networks for large-scale image recognition," 2014, *arXiv:1409.1556*.
- [16] O. Russakovsky *et al.*, "ImageNet large scale visual recognition challenge," *Int. J. Comput. Vis.*, vol. 115, pp. 211–252, Dec. 2015.
- [17] S. Gupta, A. Agrawal, K. Gopalakrishnan, and P. Narayanan, "Deep learning with limited numerical precision," in *Proc. Int. Conf. Mach. Learn.*, 2015, pp. 1737–1746.
- [18] K. Abdelouahab, M. Pelcat, J. Serot, and F. Berry, "Accelerating CNN inference on FPGAs: A survey," May 2018, *arXiv:1806.01683v1*.
- [19] K. Wang, Z. Liu, Y. Lin, J. Lin, and S. Han, "HAQ: Hardware-aware automated quantization with mixed precision," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2019, pp. 8604–8612.
- [20] H. T. Kung, "Why systolic arrays?," *IEEE Comput.*, vol. 15, no. 1, pp. 37–46, Jan. 1982.
- [21] A. Shawahna, S. M. Sait, and A. El-Maleh, "FPGA-based accelerators of deep learning networks for learning and classification," *IEEE Access*, vol. 7, pp. 7823–7859, 2019.
- [22] X. Wei *et al.*, "Automated systolic array architecture synthesis for high throughput CNN inference on FPGAs," in *Proc. 54th Annu. Des. Automat. Conf.*, 2017, pp. 1–6.
- [23] N. P. Jouppi *et al.*, "In-datacenter performance analysis of a tensor processing unit," in *Proc. 44th Int. Symp. Comput. Archit.*, 2017, pp. 1–12.
- [24] Xilinx, "Xilinx ML suite," 2019. [Online]. Available: <https://github.com/Xilinx/ml-suite>
- [25] DPU for Convolutional Neural Network V3.2-DPU IP Product Guide (PG338), Xilinx, San Jose, CA, USA, 2020.
- [26] Y. Fu, E. Wu, A. Sirasao, S. Attia, K. Khan, and R. Witting, "Deep learning with INT8 optimization on Xilinx devices," *Xilinx White Paper*, WP486, Nov. 2016.
- [27] T. Han *et al.*, "Convolutional neural network with INT4 optimization on Xilinx devices," *Xilinx White Paper*, WP521, Jun. 2020.
- [28] J. Liang, J. Han, and F. Lombardi, "New metrics for the reliability of approximate and probabilistic adders," *IEEE Trans. Comput.*, vol. 62, no. 9, pp. 1760–1771, Sep. 2013.
- [29] M. Masadeh, O. Hasan, and S. Tahar, "Input-conscious approximate multiply-accumulate (MAC) unit for energy-efficiency," *IEEE Access*, vol. 7, pp. 147129–147142, Oct. 2019.
- [30] I. Qiqieh, R. Shafik, G. Tarawneh, D. Sokolov, and A. Yakovlev, "Energy-efficient approximate multiplier design using bit significance-driven logic compression," in *Proc. Des. Automat. Test Eur. Confe. Exhib.*, 2017, pp. 7–12.
- [31] N. V. Toan and J.-G. Lee, "Energy-area-efficient approximate multipliers for error-tolerant applications on FPGA," in *Proc. 32nd IEEE Int. Syst.-on-Chip Conf.*, 2019, pp. 336–341.
- [32] S. Ullah, H. Schmidl, S. S. Sahoo, S. Rehman, and A. Kumar, "Area optimized accurate and approximate softcore signed multiplier architectures," *IEEE Trans. Comput.*, vol. 70, no. 3, pp. 384–392, Mar. 2021.
- [33] S. Venkatachalam and S.-B. Ko, "Design of power and area efficient approximate multipliers," *IEEE Trans. Very Large Scale Integr. Syst.*, vol. 25, no. 5, pp. 1782–1786, May 2017.
- [34] M. S. Ansari, H. Jiang, B. F. Cockburn, and J. Han, "Low-power approximate multipliers using encoded partial products and approximate compressors," *IEEE J. Emerg. Sel. Top. Circuits Syst.*, vol. 8, no. 3, pp. 404–416, Sep. 2018.
- [35] S. Han, H. Mao, and W. J. Dally, "Deep compression: Compressing deep neural networks with pruning, trained quantization and Huffman coding," 2015, *arXiv:1510.00149*.



Ercan Kalali received the BS degree in electronics engineering from Istanbul Technical University, Istanbul, Turkey, in 2011, and the MS and PhD degrees in electronics engineering from Sabanci University, Istanbul, Turkey, in 2013 and 2018, respectively. He is currently a postdoctoral researcher with the Circuit and Systems Group, Delft University of Technology (TU Delft), The Netherlands. His research interests include low-power digital hardware design, approximate computing, sensor fusion, deep learning, video coding, and high-level synthesis.



Rene van Leuken received the MSc and PhD degrees in electrical engineering from the Delft University of Technology, Delft, The Netherlands, in 1983 and 1988, respectively. He is currently a professor with the Circuit and Systems Group, Faculty of Electrical Engineering, Mathematics and Computer Science, Delft University of Technology (TU Delft), The Netherlands. His research interests include high-level digital system design, system design optimization, VLSI design, and high-performance compute (DSP) engines. His major research activity is neuromorphic computing. He has been involved over the years in the inception, creation, and execution of many major research and development projects: JESSI, MEDEA, ENIAC/CATRENE, ARTEMIS and recently in ECSEL and PENTA projects. He is a member of the PATMOS steering committee and the DATE Technical Program Committee. He has published papers in all major journals, conferences, and workshop proceedings and has received several best paper awards more than the years.

► For more information on this or any other computing topic, please visit our Digital Library at www.computer.org/csdl.